

Embedded Systems Project: Autonomous Robotic Car Challenge

Project Overview

This project involves the design, development, integration, and testing of an autonomous robotic car capable of navigating a predefined course. The robotic car must operate independently using onboard sensors and embedded software without remote control. It must use micro T-Kernel RTOS and run on the Raspberry Pi Pico + Robo Pico platform. It must be in C-language and use Pico C SDK. The project must also follow the template Github repo and conform to Barr C Coding Standard.

The course consists of:

- A line-following track
- Directional barcodes placed at selected locations
- Speed humps of varying heights
- Static obstacles placed along the track

The robotic car must navigate the course while collecting environmental information and making autonomous decisions based on sensor inputs.

The project emphasises:

- Embedded systems design
- Sensor integration
- Resource-efficient programming
- Real-time decision making
- Autonomous navigation
- Hardware-software co-design

As embedded systems are inherently resource-constrained, teams are expected to design efficient software and make prudent use of processing, memory, communication, and power resources.

Performance assessment will consider:

- Navigation accuracy
- Completion time
- Reliability
- Code quality
- Resource efficiency
- System robustness

Mission Requirements

The robotic car shall:

1. Follow a line track autonomously.
2. Detect and decode barcodes positioned along the track.
3. Execute navigation commands encoded by barcodes:
 - Turn Left
 - Turn Right
 - Go Straight
 - U-Turn
4. Detect humps encountered along the route.
5. Measure and report the highest hump peak experienced during the run.
6. Detect obstacles located on or near the line.
7. Profile obstacle shape and position using ultrasonic sensing.
8. Navigate around obstacles while avoiding collisions.
9. Reacquire the original line after bypassing an obstacle.
10. Report telemetry data through WiFi communication.

Expected System Behaviour

Line Following

The robot shall continuously track the black line using IR sensors. The line-following algorithm should be robust against:

- Minor lighting changes
- Slight track imperfections
- Speed variations

Barcode Navigation

Barcodes will be placed at selected locations along the route. Upon detecting a barcode, the robot shall decode the command and perform the corresponding action.

Example:

Barcode	Command
A	Turn Left
B	Turn Right
C	Go Straight
D	U-Turn

Navigation decisions must be executed accurately using encoder-based motion control.

Hump Detection

The course may contain one or more humps. The robot shall:

- Detect hump using IMU data
- Estimate hump height using IMU data

Obstacle Detection and Avoidance

Obstacles may appear directly on the line path. When an obstacle is detected:

1. The robot shall stop or slow down.
2. The ultrasonic scanning system shall profile the obstacle.
3. The robot shall determine a safe bypass route.
4. The robot shall leave the line temporarily.
5. Navigate around the obstacle.
6. Search for and reacquire the original line.
7. Continue normal operation.
8. The avoidance behaviour should minimise deviation from the original route.

Suggested System Architecture

Sensor Layer

- IR Sensors
- Wheel Encoders
- IMU
- Ultrasonic Sensor
- Barcode Sensor

Control Layer

- Motion Control
- Line Following
- Obstacle Avoidance
- Navigation Logic

Communication Layer

- WiFi
- MQTT
- Telemetry

Team Structure

General Responsibilities for **all** buddies:

- Participate in integration activities
- Participate in system testing
- Assist in debugging
- Contribute to final demonstrations
- Understand subsystem interfaces

Although each buddy owns a subsystem, the final robot is assessed as an integrated system.

Buddy 1: WiFi Communication, Command, and Telemetry

Primary Hardware: WiFi

Responsibilities: Design and implement the communication for the robotic car.

Tasks

- Establish WiFi connectivity
- Implement MQTT communication
- Design communication topics
- Define telemetry message structures
- Implement telemetry publishing
- Implement command subscription
- Implement connection recovery
- Implement heartbeat/status reporting

Telemetry Examples

- Current robot state
- Motor speed
- Encoder counts
- Line sensor status
- Barcode detected
- Hump measurements
- Obstacle scan results
- Distance travelled

Deliverables

- Communication API
- Telemetry framework
- MQTT documentation
- Demonstration of telemetry reporting

Buddy 2: Motion Control System

Primary Hardware: Motors + Wheel Encoders

Responsibilities: Develop the vehicle motion subsystem and provide reusable motion-control APIs for the rest of the team.

Tasks

- Motor driver integration
- Encoder integration
- Speed estimation
- Distance estimation
- PID speed control
- Straight-line motion correction
- Encoder-based turning
- Motion calibration

Required APIs

Examples:

- `moveForward(distance)`
- `moveBackward(distance)`
- `turnLeft(angle)`
- `turnRight(angle)`
- `stop()`

Performance Objectives

- Consistent speed
- Accurate distance travelled
- Repeatable turns
- Stable motion

Deliverables

- Motion control library
- PID tuning report
- Motion accuracy evaluation

Buddy 3: Barcode Decoding and IR Line Following

Primary Hardware: 3x IR Sensors

Responsibilities: Develop the line sensing and barcode detection subsystems.

Tasks

- IR sensor integration
- Sensor calibration
- Line position estimation
- Line-following algorithm
- Junction detection
- Barcode detection
- Barcode decoding
- Navigation command generation

Barcode Commands

- Turn Left
- Turn Right
- Go Straight

Performance Objectives

- Reliable line tracking
- Accurate barcode recognition
- Robust operation at varying speeds

Deliverables

- Line-following module
- Barcode decoder
- Navigation command interface

Buddy 4: IMU-Based Motion and Terrain Monitoring

Primary Hardware: IMU

Responsibilities: Develop the motion-monitoring and terrain-analysis subsystem.

Tasks

- IMU integration
- Sensor calibration
- Tilt detection
- Hump detection
- Peak hump measurement
- Motion classification
- Collision detection
- Turn-rate monitoring
- Motion event reporting

Example Motion Events

- Stationary
- Accelerating
- Turning
- Climbing hump
- Descending hump
- Sudden impact

Performance Objectives

- Reliable hump detection
- Accurate peak measurement
- Stable event classification

Deliverables

- IMU processing module
- Hump detection algorithm
- Terrain analysis report

Buddy 5: Adaptive Servo Ultrasonic Scanning and Obstacle Profiling

Primary Hardware: Ultrasonic Sensor + Servo

Responsibilities: Develop the obstacle sensing and avoidance subsystem.

Tasks

Stage 1: Coarse Scan

Perform an initial scan using a small number of scan angles.

Example:

- 30°
- 60°
- 90°
- 120°
- 150°

Stage 2: Fine Scan

If an obstacle is detected:

- Return to the obstacle region
- Perform higher-resolution scanning
- Improve obstacle localisation

Stage 3: Obstacle Profiling

Estimate:

- Obstacle location
- Obstacle width
- Closest point
- Left-side clearance
- Right-side clearance

Stage 4: Avoidance Planning

Determine:

- Stop
- Continue
- Turn left
- Turn right
- Reverse and reattempt

Stage 5: Line Recovery

After bypassing an obstacle:

- Search for the original line

- Reacquire line tracking
- Return control to the line-following subsystem

Performance Objectives

- Accurate obstacle detection
- Effective obstacle profiling
- Reliable avoidance
- Successful line reacquisition

Deliverables

- Scanning subsystem
- Obstacle profile generator
- Avoidance algorithm
- Recovery algorithm

Assessment Considerations

Teams will be assessed on:

- Successful completion of mission objectives
- Navigation accuracy
- Obstacle avoidance effectiveness
- Barcode recognition reliability
- Hump measurement capability
- System robustness
- Software design quality
- Resource efficiency
- Team integration quality
- Demonstration performance

Teams are encouraged to develop clean subsystem interfaces so that individual components can be developed, tested, and integrated efficiently.